

SERVIÇO DISTRIBUÍDO DE SOMA CONFIÁVEL

Nome 1 Nome 2 Nome 3 Nome 4

INF01085 – Sistemas Distribuídos e Tolerantes a Falhas

Trabalho Prático 1: comunicação confiável interprocessos — Outubro de 2026

OBJETIVO E REQUISITOS

- Receber inteiros positivos de clientes distintos e manter uma soma global.
- Usar exclusivamente UDP/IP para a comunicação entre processos.
- Garantir que cada valor entre na soma global exatamente uma vez.
- Tolerar perda, duplicação e reordenação de datagramas.
- Exibir o estado do serviço após cada requisição aceita.

Critério principal

A soma final deve corresponder à soma de todos os valores enviados por todos os clientes.

Descoberta

1. Cliente envia `DISCOVER` em broadcast.
2. Servidor registra o IP do participante.
3. Servidor responde `DISCOVER_ACK` em unicast.

Processamento

1. Cliente envia uma `REQUEST`.
2. Servidor processa ou identifica duplicata.
3. Servidor responde um `ACK`.

`Cliente A / Cliente B / ...` $\longleftrightarrow$ `Servidor UDP`

O servidor mantém somente estado em memória, como previsto para a primeira etapa.

Mensagem	Conteúdo	Tamanho
`DISCOVER`	tipo	1 byte
`DISCOVER_ACK`	tipo	1 byte
`REQUEST`	tipo, `id_req`, valor	17 bytes
`ACK`	tipo, `last_req`, `num_reqs`, `total_sum`	25 bytes
`EXIT`	tipo	1 byte

- Campos numéricos usam inteiros sem sinal de 64 bits em ordem de rede.
- O formato binário fixo reduz serialização e parsing em cargas grandes.
- O protocolo valida tamanho, tipo e faixa dos campos antes de processar uma mensagem.

1. Lê um inteiro positivo da entrada padrão.
2. Cria `REQUEST(id_req, value)` e envia ao servidor.
3. Aguarda até 100 ms por um `ACK` do servidor descoberto.
4. Aceita apenas o ACK com o mesmo `id_req` pendente.
5. Incrementa o identificador somente após a confirmação.

Timeout

Quando o limite expira, o cliente retransmite o mesmo datagrama: mesmo ID e mesmo valor. Nenhuma nova requisição é enviada nesse intervalo.

SERVIDOR: ESTADO POR CLIENTE

Tabela de participantes, indexada pelo endereço IP

Campo	Significado
`last_req`	último ID aceito
`last_num_reqs`	quantidade global no último ACK
`last_total_sum`	soma global no último ACK

Decisão para cada requisição recebida

Condição	Ação do servidor
`id_req = last_req + 1`	Soma o valor, atualiza a tabela e envia o ACK.
`id_req <= last_req`	Exibe `DUP!!` e reenvia o ACK armazenado.
`id_req > last_req + 1`	Exibe mensagem fora de ordem e devolve o último ACK.

Invariante por cliente

Depois de processar uma requisição com ID n, o servidor já aplicou exatamente uma vez as requisições de 1 até n daquele cliente.

- Apenas `last_req + 1` altera `num_reqs` e `total_sum`.
- Após essa atualização, uma nova cópia do mesmo ID cai no caso de duplicata.
- O servidor devolve o retrato do ACK original, armazenado no estado do cliente.
- O cliente mantém somente uma requisição pendente, portanto não cria lacunas no fluxo normal.

Caso de ACK perdido: o cliente retransmite, o servidor reconhece a duplicata e reenvia a confirmação sem repetir a soma.

Servidor

- Um único loop recebe e processa datagramas.
- Atualizações da tabela e do acumulador são serializadas.
- Não há duas threads alterando o estado global.

Cliente

- Uma thread lê a entrada padrão.
- Uma thread escreve mensagens na tela.
- A thread principal controla socket, ACKs e timeouts.
- Filas thread-safe transferem valores e mensagens.

Encerramento

`Ctrl+C` ou `Ctrl+D` encerra o cliente e envia `EXIT` ao servidor quando a descoberta já ocorreu.

Servidor

Inicialização

```
python3 servidor.py 4000
2026-10-09 11:00:00
num_reqs 0 total_sum 0
```

Cliente

Inicialização

```
python3 cliente.py 4000
2026-10-09 11:00:01
server_addr 10.67.103.34
```

- O cliente não mostra prompt para leitura de valores.
- Cada ACK mostra ID, valor, quantidade global e soma parcial.
- O servidor mostra cada requisição nova, duplicada ou fora de ordem.

1. Descoberta entre máquinas em rede local.
2. Envio manual, timeout e retransmissão em ambiente Ubuntu–Windows.
3. Dois clientes Linux em IPs distintos, com cargas concorrentes.
4. Validação da soma independente com `awk` antes do teste de arquivos grandes.

Critério de verificação

Para cada experimento, comparamos `num_reqs` e `total_sum` finais com a quantidade e a soma conhecidas das entradas.

RESULTADOS NO LABORATÓRIO

Teste	Requisições	Soma final	Observação
Dois clientes, 100 valores cada	200	20100	total correto
Dois clientes, 1000 valores cada	2000	2001000	sem duplicatas
`N1` e `N2`, em paralelo	2000000	51000107	sem `DUP!!`

- Cada cliente enviou 1.000.000 de valores no maior teste.
- A soma esperada veio de cálculos independentes sobre os arquivos de entrada.
- O maior teste terminou em aproximadamente 3 minutos e 12 segundos.

LIMITES DA ETAPA 1

- O estado permanece somente em memória.
- Uma falha do processo servidor elimina acumulador e tabela de participantes.
- A identificação segue o IP do cliente, conforme a especificação.
- A etapa seguinte pode acrescentar replicação e persistência consistente.

Escopo garantido

Enquanto o servidor está em execução, o protocolo trata perdas, duplicatas e reordenação de mensagens UDP sem duplicar valores no acumulador.

CONCLUSÃO

- Descoberta por broadcast identifica o servidor no segmento local.
- Stop-and-wait, identificadores e ACKs implementam confiabilidade sobre UDP.
- A tabela por cliente preserva o resultado de cada operação confirmada.
- Os testes em máquinas distintas confirmaram a soma correta sob carga concorrente.

Perguntas?